

Isomorphic coupled-task scheduling problem with compatibility constraints on a single processor

G. Simonin · B. Darties · R. Giroudeau · J.-C. König

Published online: 16 September 2010
© Springer Science+Business Media, LLC 2010

Abstract The problem presented in this paper is a generalization of the usual coupled-tasks scheduling problem in presence of compatibility constraints. The reason behind this study is the data acquisition problem for a submarine torpedo. We investigate a particular configuration for coupled tasks (any task is divided into two sub-tasks separated by an idle time), in which the idle time of a coupled task is equal to the sum of durations of its two sub-tasks. We prove $\mathcal{NP}$ -completeness of the minimization of the schedule length, we show that finding a solution to our problem amounts to solving a graph problem, which in itself is close to the minimum-disjoint-path cover (min-DCP) problem. We design a $(\frac{3a+2b}{2a+2b})$ -approximation, where a and b (the processing time of the two sub-tasks) are two input data such as $a > b > 0$, and that leads to a ratio between $\frac{3}{2}$ and $\frac{5}{4}$. Using a polynomial-time algorithm developed for some class of graph of min-DCP, we show that the ratio decreases to $\frac{1+\sqrt{3}}{2} \approx 1.37$.

Keywords Coupled-tasks · Complexity · Compatibility graph · Polynomial-time approximation

G. Simonin (✉) · R. Giroudeau · J.-C. König
LIRMM UMR 5506, rue Ada, 34392 Montpellier Cedex 5, France
e-mail: simonin@lirmm.fr

R. Giroudeau
e-mail: rgirou@lirmm.fr

J.-C. König
e-mail: könig@lirmm.fr

B. Darties
LE2I UMR 5158, 9 Rue Alain Savary, 21000 Dijon, France
e-mail: Benoit.Darties@u-bourgogne.fr

1 Introduction

In this paper, we present a scheduling problem of coupled tasks subject to compatibility constraints, which is a generalization of the scheduling problem of coupled tasks first introduced by Shapiro (1980). This problem is motivated by the problem of data acquisition in a submarine torpedo. The aim amounts to treating various environmental data coming from sensors located on the torpedo, that collect information which must be processed on a single processor. A single acquisition task can be described as follows: a sensor of the torpedo emits a wave at a certain frequency (according to the data that must be collected) which propagates in the water and reflects back to the sensor. This acquisition task is divided into two sub-tasks: the first task consists in sending an ultrasound pulse while the second receives returning echo. Between them, there is an incompressible idle time which represents the spread of the echo under the water. Thus acquisition tasks may be assigned to coupled tasks.

In order to use idle time, other sensors can send more echoes. However, the proximity of the waves causes disruptions and interferences. In order to handle information error-free, a compatibility graph between acquisition tasks is created. In this graph, which describes the set of tasks, we have an edge between two compatible tasks. A task is compatible with another if at least one of its sub-tasks can be executed during the idle time of another task. Given a set of coupled tasks and such a compatibility graph, the aim is to schedule the coupled tasks in order to minimize the time required for the completion of all the tasks.

1.1 Notations

First we present some common notations:

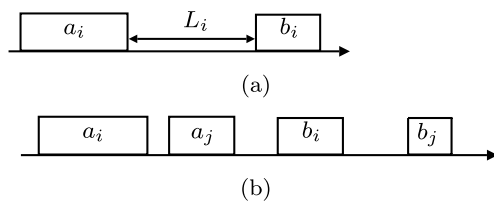


Fig. 1 A single coupled task and two compatible coupled tasks

- Let G be an undirected graph. We note $V(G)$ the set of its vertices and $E(G)$ the set of its edges;
- we note n (resp. m) the cardinality of set $V(G)$ (resp. $E(G)$);
- a *path* is a non-empty graph C with $V(C) = \{x_0, x_1, \dots, x_k\}$ and $E(C) = \{x_0x_1, \dots, x_{k-1}x_k\}$, where all the x_i are distinct;
- the length of a path is the number of edges that the path uses.

Then, we introduce the notations relative to coupled tasks we will use in the rest of the paper: we note $\mathcal{A} = \{A_1, A_2, \dots, A_n\}$ the set of n coupled tasks. Using the notation proposed by Shapiro (1980), each task $A_i \in \mathcal{A}$ is composed of two sub-tasks a_i and b_i . For clarity we use the same notations for the processing time of these tasks: a_i and b_i have processing time $a_i \in \mathbb{N}$ and $b_i \in \mathbb{N}$, and separated by a fixed idle time $L_i \in \mathbb{N}$ (see Fig. 1(a)). For each i the second sub-task b_i must start its execution exactly L_i time units after the completion time of a_i .

According to the torpedo problem, a task may be started during the idle time of a running task if it uses another frequency, is not dependant on the execution of the running task (and reciprocally), or does not require to access the resources used by the running tasks. Formally, we say that two tasks A_i and A_j are *compatible* if and only if we can execute at least a sub-task of A_i during the idle time of A_j (see Fig. 1(b)). On the other side, some tasks cannot be compatible due to previously cited reasons.

1.2 Main problem formulation

We aim at scheduling a set of coupled tasks with compatibility constraints on a monoprocessor. The input of the general problem is described with the set $\mathcal{A} = \{A_1, A_2, \dots, A_n\}$ of coupled tasks and a compatibility graph G_c , with $V(G_c) = \mathcal{A}$ and $E(G_c)$ the edges which represent all pairs of compatible tasks, i.e. an edge exists between A_i and A_j if and only if a_j can be scheduled between a_i and b_i (Fig. 1(b)). Note that compatibility is symmetrical, thus here a_i could be scheduled between a_j and b_j .

The solution of an instance consists in determining the starting time of each sub-task a_i of each task $A_i \in \mathcal{A}$. The tasks have to be processed on a single processor while preserving the constraints given by the compatibility graph

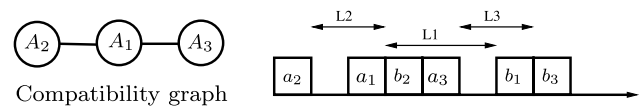


Fig. 2 Link between the compatibility graph and the scheduling

(see Fig. 2). Formally, we need to find a *valid schedule* $\sigma : \mathcal{A} \rightarrow \mathbb{N}$ where the notation $\sigma(A_i)$ denotes the starting time of the task A_i . We use the following abuse of notation: $\sigma(a_i) = \sigma(A_i)$ (resp. $\sigma(b_i) = \sigma(A_i) + a_i + L_i$) denotes the starting time of the first sub-task a_i (resp. the second sub-task b_i).

Let $C_{\max} = \max_{A_i \in \mathcal{A}} (\sigma(A_i) + a_i + L_i + b_i)$ be the required time to complete all the tasks. Then the objective is to find a feasible schedule which minimizes $C_{\max}$. We use the notation scheme $\alpha|\beta|\gamma$ proposed by Graham et al. (1979), where α denotes the environment processors, β the characteristics of the jobs and γ the criteria. The main problem denoted as Π will be defined by:

$$\Pi = 1|coupled-task, (a_i, b_i, L_i), G_c|C_{\max}.$$

1.3 Related work

The problem of coupled tasks has been studied in regard to different conditions on the values of a_i, b_i, L_i for $1 \leq i \leq n$, and precedence constraints (Błażewicz et al. 2009; Ahr et al. 2004; Lehoux-Lebacque et al. 2009; Orman and Potts 1997). Note that, in the previous works, all tasks are compatible by considering a complete graph (Błażewicz et al. 2009; Ahr et al. 2004; Lehoux-Lebacque et al. 2009; Orman and Potts 1997). Moreover, in presence of any compatibility graph, we find several complexity results (Simonin 2009; Simonin et al. 2009a; Simonin et al. 2009b), which are summarized in Table 1. The notation $a_i = a$ implies that for all $1 \leq i \leq n$, a_i is equal to a constant $a \in \mathbb{N}$. This notation can be extended to b_i and L_i with the constants b, L and $p \in \mathbb{N}$.

1.4 Contribution and organization of this paper

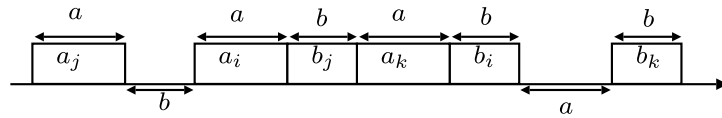
Our work consists in measuring the impact of the compatibility graph on the complexity and approximation of scheduling problems with coupled tasks on a monoprocessor. In this way, we focus our work on establishing the limits between polynomiality and $\mathcal{NP}$ -completeness of these problems according to some parameters, when the compatibility constraints is introduced. In (Simonin et al. 2009a; Simonin et al. 2009b), we have studied the impact of the parameter L , and have shown that the problem $1|coupled-task, (a_i = b_i = p, L_i = L), G_c|C_{\max}$ was $\mathcal{NP}$ -complete as soon as $L \geq 2$, and polynomial otherwise.

In this work, we complete complexity results with the study of other special cases according to the value of a_i

Table 1 Complexity for scheduling problems with coupled-tasks and compatibility constraints

Problem	Complexity	Ref.
1 coupled-task, $(a_i = b_i = L_i), G_c C_{\max}$	$\mathcal{NP}$ -complete	Simonin (2009)
1 coupled-task, $(a_i = a, b_i = b, L_i = L), G_c C_{\max}$	$\mathcal{NP}$ -complete	Simonin (2009)
1 coupled-task, $(a_i = b_i = p, L_i = L), G_c C_{\max}$	$\mathcal{NP}$ -complete	Simonin et al. (2009b)
1 coupled-task, $(a_i = L_i = p, b_i), G_c C_{\max}$	$O(n^2m)$	Simonin (2009)
1 coupled-task, $(a_i, b_i = L_i = p), G_c C_{\max}$	$O(n^2m)$	Simonin (2009)

Fig. 3 At most 2 sub-tasks may be scheduled between a_i and b_i



and b_i , and we propose several approximation algorithms for them. We restrict our study to a special case, by adding new hypotheses to the processing time and idle time of the tasks. For any task A_i , $i \in \{1, \dots, n\}$, the processing time a_i (resp. b_i) of sub-task a_i (resp. b_i) is equal to a constant a (resp. b), and the length of the idle time between a_i and b_i is L . Considering homogeneous tasks is a realistic hypothesis according to the tasks that the torpedo has to execute. Let Π_1 be this new problem. Formally:

$$\Pi_1 = 1|\text{coupled-task}, (a_i = a, b_i = b, L_i = L), G_c|C_{\max}.$$

This paper is organized as follows: in Sect. 2, we establish the complexity of Π_1 according to the values of a , b and L , and we show that the problem is polynomial for any $L < a + b$; then we consider in the rest of the paper that $L = a + b$. In that case, the problem can be considered as a new graph problem we call MINIMUM SCHEDULE-LINKED DISJOINT-PATH COVER (MIN-SLDPC). We present the proof of $\mathcal{NP}$ -completeness of MIN-SLDPC and we conclude this section by the study of a specific sub-case with $a = b = L/2$. In Sect. 3, we show that MIN-SLDPC is immediately 2-approximated by a simple approach: we design a polynomial-time approximation algorithm with performance guarantee lower than $\frac{3}{2}$. In fact, we show that the approximation ratio obtained by this algorithm is between $\frac{3}{2}$ and $\frac{5}{4}$, according to the values of a and b . The last section is devoted to the study of Π_1 for some particular topology of the graph G_c . First we present a well-known graph problem, MINIMUM DISJOINT-PATH COVER, (MIN-DPC). Then we show the relation between MIN-DPC and MIN-SLDPC and evaluate how results from the first one can be applied to solve the second problem on specific topologies. This implies the reduction of the performance ratio we can obtain on some restricted instances from $\frac{3}{2}$ to ≈ 1.37 .

2 Computational complexity

First, we prove that Π_1 is polynomial when $L < a + b$: it is obvious that a maximum matching in the graph G_c gives

an optimal solution. Indeed, during the idle time L of a coupled task A_i , we can process at most one sub-task a_j or b_k . Since the idle time L is identical, so it is obvious that finding an optimal solution consists in computing a maximum matching. Thus, the problem 1|coupled-task, $(a_i = a, b_i = b, L_i = L < a + b), G_c|C_{\max}$ admits a polynomial-time algorithm with complexity $O(m\sqrt{n})$ where n is the number of tasks and m the number of edges of G_c (see Schrijver 2004).

The rest of the paper is devoted to the case $L = a + b$. Without loss of generality, we consider the case¹ of $b < a$. The particular case $b = a$ will be discussed in Sect. 2.2.

2.1 From a scheduling problem to a graph problem

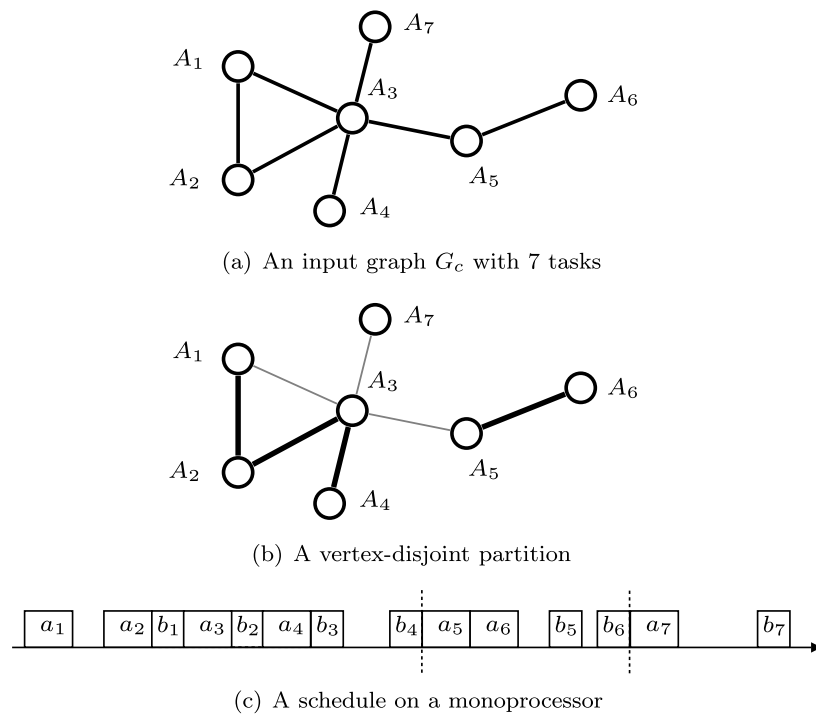
Let us consider a valid schedule σ of an instance $(\mathcal{A}, G_c)$ of Π_1 with $b < a$, composed of a set of coupled tasks $\mathcal{A}$ and a compatibility graph G_c . For a given task A_i , at most two sub-tasks may be scheduled between the completion time of a_i and the starting time of b_i , and in this case the only available schedule consists in executing a sub-task b_j and a sub-task a_k during the idle time L_i with $i \neq j \neq k$ such that $\sigma(b_j) = \sigma(a_i) + a$ and $\sigma(a_k) = \sigma(a_i) + a + b$. Figure 3 shows a such configuration.

We can conclude that any valid schedule σ can be viewed as a partition $\{T_1, T_2, \dots, T_k\}$ of $\mathcal{A}$, such that for any T_i the subgraph $P_i = G_c[T_i]$ of G_c induced by vertices T_i is a path (here, isolated vertices are considered as paths of length 0). Clearly, $\{P_1, P_2, \dots, P_k\}$ is a partition of G_c into vertex-disjoint paths. Figure 4 shows an instance of Π_1 (Fig. 4(a)), a valid schedule (Fig. 4(c))—not necessarily an optimal one—and the corresponding partition of G_c into vertex-disjoint paths (Fig. 4(b)).

For a given feasible schedule σ , let us analyze the relation between the length of the schedule $C_{\max}$ and the corresponding partition $\{P_1, P_2, \dots, P_k\}$ into vertex-disjoint

¹The results we present here can be symmetrically extended to instances with $b > a$.

Fig. 4 Relation between a schedule and a partition into vertex-disjoint paths



paths. Clearly, we have $C_{\max} = t_{\text{seq}} + t_{\text{idle}}$ where $t_{\text{seq}} = n(a+b)$ and t_{idle} is the inactivity time of the processor. Since t_{seq} is fixed for a given instance, t_{idle} obviously depends on the partition. We propose the following lemma:

Lemma 1 Let $\{P_1, P_2, \dots, P_k\}$ be the partition of vertex-disjoint paths corresponding to a schedule σ .

1. A path of length 0 corresponds to a single task scheduled in σ , t_{idle} is incremented by $L = a + b$;
2. for any path of length 1, t_{idle} is increased by a ;
3. for any path of length strictly greater than 1, t_{idle} is incremented by $(a + b)$.

Proof Point 1 is obvious. Figure 5 illustrates points 2 and 3: a path of length 1 represents 2 tasks that may be imbricated as on Fig. 5(a). Paths of length strictly greater than 1 represent more than two tasks. These tasks can be scheduled in order to get an idle time of length b at the beginning of the schedule and one of length a at the end of it (as on Fig. 5(b)). The reader could check there is no other way to imbricate tasks in order to reduce the idle time for paths of any length. $\square$

Thus, there exists a link between finding an optimal schedule and a graph problem which is called MINIMUM SCHEDULE-LINKED DISJOINT-PATH COVER (MIN-SLDPC) defined in Table 2: Clearly, MIN-SLDPC is equivalent to Π_1 with $b < a$ and $L = a + b$, and can be viewed as the graph problem formulation of a scheduling

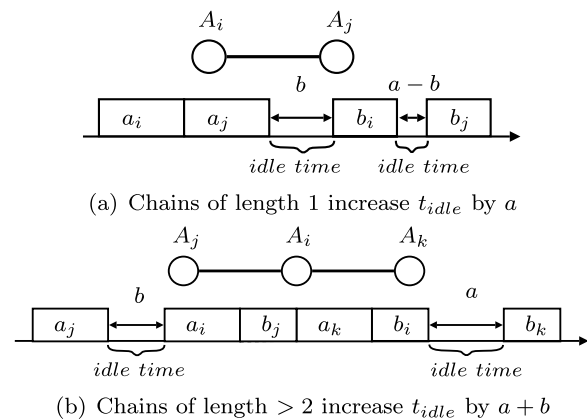


Fig. 5 Impact of the length of the paths on the idle time

Table 2 MINIMUM SCHEDULE-LINKED DISJOINT-PATH COVER (MIN-SLDPC)

Instance: a graph $G = (V, E)$ of order n , two natural integers a and b , $b < a$.

Result: a partition $\mathcal{P}$ of G into vertex-disjoint paths (can be of length 0)

Objective: Minimize $n(a + b) + \sum_{p \in \mathcal{P}} w(p)$ where $w : \mathcal{P} \rightarrow \mathbb{N}$ is a cost function with $w(p) = a$ if and only if $|E(p)| = 1$, and $w(p) = a + b$ otherwise.

problem. In any solution, each path increments the cost of idle time by at least a (when the path has a length 1), and at most $a + b < 2a$. So, we can deduce that an optimal solution to MIN-SLDPC consists in finding a partition $\mathcal{P}$ with a particular cardinality k^* , and a maximal number of paths

of length 1 among all possible k^* -partitions. The following immediate theorem establishes the complexity of MIN-SLDPC:

Theorem 1 MIN-SLDPC is an $\mathcal{NP}$ -hard problem.

Proof We consider the decision problem associated to MIN-SLDPC. We will prove that the problem of deciding whether an instance of SLDPC has a schedule of length at most $(n+1)(a+b)$ is $\mathcal{NP}$ -complete. Our proof is based on the polynomial-time transformation HAMILTONIAN PATH $\propto$ SLDPC. We keep the graph and the vertices is the task to schedule. Let us consider a graph G .

This transformation can be clearly computed in polynomial time.

- Assume that the length of the optimal schedule is $C_{\max}^{\text{opt}} = (n+1)(a+b)$. We will prove that the graph G possess a Hamiltonian path i.e. $k = 1$. Recall first that k is the number of partition and that $b < a$. We know, from the previous discussion, that $t_{\text{seq}} = n(a+b)$ and that a chain of length one increase t_{idle} by a (see illustration given by Figs. 5(b) and 5(a)). It is clear that the graph G must be covered by paths of different lengths.

Suppose that the graph G is covered by k paths with k_1 paths of length one (the set of these paths is denoted by P_1), and k_2 paths of length greater than one (resp. by $P_{\geq 2}$).

So the length of schedule given by this covering is:

$$\begin{aligned} C_{\max}^h &= \overbrace{n(a+b)}^{\text{processing times}} + \overbrace{a \times k_1}^{\text{idle time for } P_1} + \overbrace{(a+b)k_2}^{\text{idle time for } P_{\geq 2}} \\ &= (n+k_2)(a+b) + k_1a > C_{\max}^* \\ &\text{if } (k_1 \neq 0 \text{ and } k_2 \geq 1) \text{ or } (k_1 > 1 \text{ and } k_2 = 0). \end{aligned}$$

Thus, the only schedule requiring exactly $(n+1)(a+b)$ units of time implies that the graph possess a Hamiltonian path i.e. $k_1 = 0$ and $k_2 = 1$.

- Reciprocally, we suppose that the graph G possess a Hamiltonian path, we will prove the existence of a schedule of length $C_{\max} = (n+1)(a+b)$.

G contains an Hamiltonian path, we can deduce a schedule with $C_{\max} = (n+1)(a+b)$: as $t_{\text{seq}} = n(a+b)$, t_{idle} must be equal to $(a+b)$, which is possible if and only if the schedule is represented with only one chain. $\square$

2.2 A particular case $\Pi_2 : 1|\text{coupled-task}, (a_i = b_i = p, L_i = L = 2p), G_c|C_{\max}$

In this subsection only, we suppose that both sub-tasks are equal to a constant p and that the inactivity time is equal to a constant $L = 2p$.

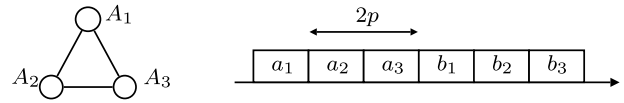


Fig. 6 Illustration of a schedule without idle time

The previous proof cannot be used for this case. Indeed the structure of these tasks allows to schedule three compatible tasks together without idle time (see Fig. 6). Another solution consists in covering vertices of G_c by triangles and paths (length 0 allowed), where we minimize the number of paths and then maximize the number of path of length 1.

This problem is a generalization of TRIANGLE PACKING (Garey and Johnson 1979) since an optimal solution without idle time consists in partitioning into triangles the vertices of G_c . This problem is well known to be $\mathcal{NP}$ -complete, and leads to the $\mathcal{NP}$ -completeness of problem Π_2 .

A correct approximation algorithm for this problem is an algorithm close to the general case. Indeed, finding an optimal solution to this problem amounts to finding a covering of the graph G_c by triangles and paths, which minimize the idle time. In the following section, we will develop an efficient polynomial-time approximation algorithm for the general problem MIN-SLDPC.

3 Approximation algorithm for MIN-SLDPC

Notice that the following algorithm, executing sequentially the tasks, admits a ratio equal to two.

We also develop a polynomial-time $\frac{3}{2}$ -approximation algorithm based on a maximum matching in the graph G_c . In fact, we show that this algorithm has an approximation ratio of at most $\frac{3a+2b}{2a+2b}$, which leads to a ratio between $\frac{3}{2}$ and $\frac{5}{4}$ according to the values of a and b (with $b < a$). This result, which depends on the values of a and b , will be discussed in Sect. 4, in order to propose a better ratio on some class of graphs.

For any instance of MIN-SLDPC, an optimal schedule has a length $C_{\max}^{\text{opt}} = t_{\text{seq}} + t_{\text{idle}}^{\text{opt}}$ where $t_{\text{seq}} = n(a+b)$.

Remark 1 For any solution of length $C_{\max}$, we necessarily have² $t_{\text{idle}} \geq (a+b)$ and also³ $t_{\text{idle}} \leq n(a+b)$. Then, for any solution h of MIN-SLDPC we have a performance ratio

²The equality is obtained when the graph G_c possesses an hamiltonian path, otherwise we need at least two paths to cover G_c (where G_c is not only an edge), which leads to increase t_{idle} by at least $2a \geq a+b$ units of time.

³The worst case consists in executing tasks sequentially without scheduling any sub-task a_j or b_j of task A_j during the idle time of a task A_i .

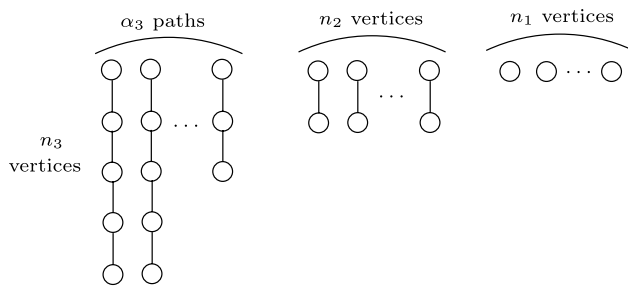


Fig. 7 Illustration of the optimal solution on an instance I

$\rho(h)$ such that:

$$\rho(h) \leq \frac{C_{\max}^h}{C_{\max}^{\text{opt}}} \leq \frac{2n(a+b)}{(n+1)(a+b)} < 2. \quad (1)$$

Indeed, we have $C_{\max}^{\text{opt}} \geq T_{\text{seq}} = n(a+b) + (a+b)$.

In the following, we develop a polynomial-time approximation algorithm based on a maximum matching in the graph G_c , with performance guaranty in $[\frac{5}{4}, \frac{3}{2}]$ according to the values of a and b .

Let I be an instance of our problem. An optimal solution is a disjoint-paths cover. The n vertices are partitioned in three disjoint sets: n_1 uncovered vertices, n_2 vertices covered by $\alpha_2 = \frac{n_2}{2}$ paths of length 1, and n_3 vertices covered by exactly α_3 paths of length strictly greater than 1 (see illustration Fig. 7). The cost of an optimal solution is equal to the sum of sequential time and idle time:

$$\begin{aligned} C_{\max}^{\text{opt}} = & \underbrace{n(a+b)}_{\text{processing times}} + \underbrace{\frac{n_2}{2}a}_{\text{idle time for matched vertices}} \\ & + \underbrace{(a+b)n_1}_{\text{idle time for isolated vertices}} \\ & + \underbrace{(a+b)\alpha_3}_{\text{idle time for path of length } > 1}. \end{aligned}$$

Now, we propose a polynomial-time approximation algorithm with non trivial ratio on an instance I . This algorithm is based on a maximum matching in G_c in order to process two coupled tasks at a time. For two coupled tasks A_i and A_j connected by an edge of the matching, we obtain an idle time of length a (see Fig. 5(a)).

Let M^* be the cardinality of a maximum matching. In the worst case, the α_3 paths are all odd and a matching of the paths leaves α_3 isolated vertices. So, we have by hypothesis:

$$M^* \geq \frac{n_2}{2} + \left(\frac{n_3}{2} - \alpha_3 \right) = \Gamma \quad (\text{worst case}). \quad (2)$$

Indeed, based on the decomposition given by the Fig. 7, we may deduce a matching within this cardinality: the α_2

paths of length one are contained in the matching; for any path of length greater than one, we include odd edges to the matching. In the worst case, all α_3 paths have odd length and the number of uncovered nodes is α_3 . If the tasks of the matching are processed first and the isolated vertices in second, the length of schedule is:⁴

$$C_{\max}^h \leq \underbrace{n(a+b)}_{\text{processing times}} + \underbrace{a \times \Gamma}_{\text{idle time 1}} + \underbrace{(a+b)(\alpha_3 + n_1)}_{\text{idle time 2}}.$$

Since an optimal length is $C_{\max}^{\text{opt}} = n(a+b) + n_1(a+b) + \frac{n_2}{2}a + \alpha_3(a+b)$, we obtain, with the last equation involved $C_{\max}^h$, the following ratio of the polynomial-time approximation algorithm:

$$C_{\max}^h \leq C_{\max}^{\text{opt}} + \left(\frac{n_3 - \alpha_3}{2} \right) a,$$

$$\rho(h) \leq 1 + \frac{(\frac{n_3 - \alpha_3}{2})a}{n(a+b) + n_1(a+b) + \frac{n_2}{2}a + \alpha_3(a+b)},$$

$$\rho(h) \leq 1 + \frac{(\frac{n_3 - \alpha_3}{2})a}{(n + n_1 + \alpha_3)(a+b) + \frac{n_2}{2}a},$$

$$\rho(h) \leq 1 + \frac{\frac{n_3}{2}a}{(n + n_1)(a+b) + \frac{n_2}{2}a},$$

max obtained for $\alpha_3 = 0$,

$$\rho(h) \leq 1 + \frac{\frac{n_3}{2}a}{n(a+b)} \leq 1 + \frac{\frac{n}{2}a}{n(a+b)}, \quad \text{since } n_3 \leq n,$$

$$\rho(h) \leq 1 + \frac{a}{2(a+b)} = \frac{3a+2b}{2a+2b}.$$

4 Instances with particular topologies

We conclude this work by a study of MIN-SLDPC when G_c admits a particular topology. First we present a related problem: MINIMUM DISJOINT PATH COVER PROBLEM (MIN-DPC). This problem has some interesting results on restricted topologies. We establish a link between MIN-DPC and MIN-SLDPC and we show that finding a ρ_{dpc} -approximation for MIN-DPC on G_c allows to find a strategy with performance ratio $\rho_{\text{slldpc}} \leq \min\{\rho_{\text{dpc}} \times (\frac{a+b}{a}), \frac{3a+2b}{2a+2b}\}$. This leads to propose, independently from the values a and b , a $\frac{1+\sqrt{3}}{2}$ -approximation for MIN-SLDPC when MIN-DPC can be polynomially solved on G_c .

4.1 A related problem: MIN-DPC

The graph problem MIN-SLDPC is very close to the well-known problem MINIMUM DISJOINT PATH COVER (MIN-DPC) which consists in covering the vertices of a graph with

⁴Idle time 1 (resp. 2) represents the idle time for matched vertices (resp. isolated vertices).

a minimum number of vertex-disjoint paths.⁵ This problem has been studied in depth in several graph classes: it is known that this problem is polynomial on cographs (Nakano et al. 2003), blocks graphs and bipartite permutation graphs (Srikant et al. 1993), distance-hereditary graph (Hung and Chang 2007), and on interval graphs (Arikati and Rangan 1990). Boesch et al. (1974) and Goodman et al. (1975) have proposed (independently) a polynomial-time algorithm in the case where the graph is a tree. Few years later, Kundu (1976) showed that this algorithm can be implemented in linear time. Among the other results, there is a polynomial-time algorithm for the cacti (Moran and Wolfstahl 1988), and another for the line graphs of a cactus (Detti and Meloni 2004). In circular-arc graphs Hung and Chang (2006) have proposed an approximation algorithm of complexity $O(n)$, which returns an optimal number of paths to a nearly constant additive equal to 1.

The problem MIN-DPC is directly linked to HAMILTONIAN COMPLETION (Garey and Johnson 1979), which consists in finding the minimum number of edges, noted $HC(G)$, that must be added to a given graph G , in order to make it hamiltonian (to guarantee the existence of a Hamiltonian cycle). It is known that if G is not hamiltonian, then the cardinality of a minimum-disjoint-path cover is clearly equal to $HC(G)$.

The dual of MIN-DPC is MAXIMUM DISJOINT-PATH COVER (Garey and Johnson 1979). It consists in finding in G a collection of vertex-disjoint paths of length at least 1, which maximises the edges covered in G . This problem is known to be $\frac{7}{6}$ -approximable (Berman and Karpinski 2006).

4.2 Relation between MIN-DPC and MIN-SLDPC

From the literature, we know that MIN-DPC is polynomial on trees (Goodman et al. 1975; Kundu 1976; Boesch et al. 1974), distance-hereditary graphs (Hung and Chang 2007), bipartite permutation graphs (Srikant et al. 1993), cactis (Moran and Wolfstahl 1988) and many others classes. There are currently no result about the complexity of MIN-SLDPC on such graphs: since the values of a and b have a high impact, techniques used to prove the polynomiality of MIN-DPC cannot be adapted to prove the polynomiality of MIN-SLDPC. Despite all our effort, the complexity of MIN-SLDPC remains an open problem. However using known results on MIN-DPC, we show how the approximation ratio can be decreased for these class of graphs. We propose the following lemma:

Lemma 2 *If MIN-DPC can be solved polynomial computation time, then there exists a polynomial-time $\frac{(a+b)}{a}$ -approximation for MIN-SLDPC.*

⁵Sometimes referenced as the PATH-PARTITION problem (PP).

Proof Let $I_1 = (G)$ be an instance of MIN-DPC, and $I_2 = (G, a, b)$ an instance of MIN-SLDPC. Let $\mathcal{P}_1^*$ be an optimal solution of MIN-DPC of cost $|\mathcal{P}_1^*|$, and $\mathcal{P}_2^*$ an optimal solution of MIN-SLDPC of cost OPT_{sldpc} . According to the definition of MIN-SLDPC, we have $|\mathcal{P}_2^*| \geq |\mathcal{P}_1^*|$.⁶ Since each path of a MIN-SLDPC solution increments the cost of the solution by at least a , then we have:

$$\begin{aligned} OPT_{\text{sldpc}} &= \sum_{p \in \mathcal{P}_2^*} w(p) + n(a+b) \\ &\geq a|\mathcal{P}_2^*| + n(a+b) \geq a|\mathcal{P}_1^*| \end{aligned} \quad (3)$$

$$\Rightarrow \frac{OPT_{\text{sldpc}}}{a} \geq |\mathcal{P}_1^*|. \quad (4)$$

Let us consider the partition $\mathcal{P}_1^*$ as a solution (not necessarily optimal) to the instance I_2 of MIN-SLDPC, and let us evaluate it cost. Since each path of a MIN-SLDPC solution increments the cost of the solution by at most $a+b$, then we have:

$$\begin{aligned} \sum_{p \in \mathcal{P}_1^*} w(p) + n(a+b) &\leq (a+b)|\mathcal{P}_1^*| + n(a+b) \\ &\leq (a+b)|\mathcal{P}_2^*| + n(a+b) \\ &\leq b|\mathcal{P}_2^*| + OPT_{\text{sldpc}} \quad \text{according to (3)} \\ &\leq \frac{b}{a}OPT_{\text{sldpc}} + OPT_{\text{sldpc}} \quad \text{according to (4)} \\ &\leq \frac{a+b}{a}OPT_{\text{sldpc}}. \end{aligned} \quad (5)$$

□

The same proof may be applied if there exists a ρ_{dpc} -approximation for MIN-DPC, and then there exists a $\rho_{\text{dpc}} \times (\frac{a+b}{a})$ -approximation for MIN-SLDPC. Let us suppose that we know a constant ρ_{dpc} such that there exists a ρ_{dpc} -approximation for MIN-DPC on G_c . Let S_1 be the strategy, which consists in determining a $\rho_{\text{dpc}} \times (\frac{a+b}{a})$ -approximation for MIN-SLDPC from ρ_{dpc} , and S_2 the strategy, which consists in using the algorithm introduced in Sect. 3. Clearly, S_1 is particularly relevant when b is very small in comparison with a . Whereas S_2 gives better ratio when b is close to a . Both strategies are complementary along the value of b , which varies from 0 to a . Choosing the best result between the execution of S_1 and S_2 , gives a performance ratio ρ_{sldpc} such that:

$$\rho_{\text{sldpc}} \leq \min \left\{ \rho_{\text{dpc}} \times \left(\frac{a+b}{a} \right), \frac{3a+2b}{2a+2b} \right\}. \quad (6)$$

⁶The best solution for MIN-SLDPC is not necessarily a solution with a minimum cardinality of k .

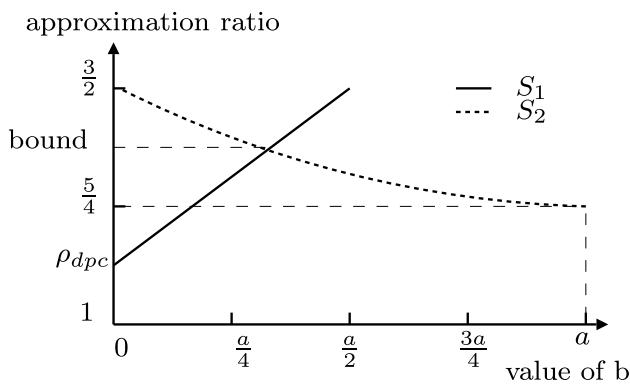


Fig. 8 Finding a good ρ_{dpc} -approximation helps to increase the results of Sect. 3

Compared to executing S_1 only, this new strategy increases the obtained results if and only if ρ_{dpc} is lower than $\frac{3}{2}$ (see Fig. 8).

We propose the following remark, which is not good news:

Remark 2 There is no ρ_{dpc} -approximation for MIN-DPC in general graphs for some $\rho_{dpc} < 2$.

This result is a consequence of the Impossibility Theorem (Chrétienne and Picouleau 1995). It can be checked by considering an instance of MIN-DPC which has an hamiltonian path: the optimal solution of MIN-DPC has cost 1, thus any polynomial-time ρ_{dpc} -approximation algorithm with $\rho_{dpc} < 2$ will return a solution of cost 1, which is not allowed under the assumption that $\mathcal{P} \neq \mathcal{NP}$. This result also implies that constant-factor approximation algorithms for max-DPC do not necessarily give the same performance guarantees on min-DPC, since the best approximation ratio for max-DPC is $\frac{7}{6}$, which is lower than the inapproximability bound for min-DPC.

It is good news that MIN-DPC is polynomial for some compatibility graphs such as trees (Goodman et al. 1975; Kundu 1976; Boesch et al. 1974), distance-hereditary graphs (Hung and Chang 2007), bipartite permutation graphs (Srikant et al. 1993), cactis (Moran and Wolfstahl 1988) and many others classes; thus $\rho_{dpc} = 1$. For all these graphs we obtain an approximation ratio of $\min\{\frac{(a+b)}{a}, \frac{3a+2b}{2a+2b}\}$ which is maximal when $\frac{(a+b)}{a} = \frac{3a+2b}{2a+2b}$, i.e.:

$$\frac{(a+b)}{a} = \frac{3a+2b}{2a+2b} \iff -a^2 + 2ab + 2b^2 = 0. \quad (7)$$

The only solution of this equation with a and $b \geq 0$ is $a = b(1 + \sqrt{3})$. By replacing a by this new value on $\frac{(a+b)}{a}$ or on $\frac{3a+2b}{2a+2b}$, we show that in the worst case the approximation ratio is reduced from $\frac{3}{2}$ down to $\frac{1+\sqrt{3}}{2} \approx 1.37$.

5 Conclusion

We investigate a particular coupled-tasks scheduling problem Π_1 in presence of a compatibility graph. We have shown how this scheduling problem can be reduced to a graph problem. We have proved that adding the compatibility graph leads to the $\mathcal{NP}$ -completeness of Π_1 , whereas the problem is obviously polynomial when there is a complete compatibility graph (each task is compatible with each other). We have proposed a ρ -approximation of Π_1 where ρ is between $\frac{3}{2}$ and $\frac{5}{4}$ according to value of a and b . We have also decreased the upper bound of $\frac{3}{2}$ down to ≈ 1.37 on instances where MINIMUM DISJOINT PATH COVER can be polynomially solved on the compatibility graph.

As perspectives of this work, we plan to test the pertinence of our polynomial-time approximation algorithm through simulations in order to determine the average gap between the optimal solution and the results obtained with our strategy on significant instances. We also aim to classify the complexity of other configurations, especially $\Pi_1 : 1|coupled-task, (a_i = a, b_i = b, L_i = L)|C_{\max}$ with a complete compatibility graph.

Acknowledgement Thanks to our reviewers for the thorough review of this paper and many helpful comments and suggestions.

References

- Ahr, D., Békési, J., Galambos, G., Oswald, M., & Reinelt, G. (2004). An exact algorithm for scheduling identical coupled-tasks. *Mathematical Methods of Operations Research*, 59(11), 193–203.
- Arikati, R. S., & Rangan, P. C. (1990). Linear algorithm for optimal path cover problem on interval graphs. *Information Processing Letters*, 35(3), 149–153.
- Berman, P., & Karpinski, M. (2006). 8/7-approximation algorithm for (1, 2)-TSP. In *SODA '06: Proceedings of the seventeenth annual ACM-SIAM symposium on Discrete algorithm* (pp. 641–648). New York: ACM.
- Błażewicz, J., Ecker, K., Kis, T., Potts, C. N., Tanas, M., & Whitehead, J. (2009). Scheduling of coupled tasks with unit processing times. *Technical report, Poznan University of Technology*.
- Boesch, F. T., Chen, S., & McHugh, B. (1974). On covering the points of a graph with point-disjoint paths. *Graphs and Combinatorics*, 406, 201–212.
- Chrétienne, Ph., & Picouleau, C. (1995). Scheduling with communication delays: a survey. In *Scheduling theory and its applications* (pp. 641–648). New York: Wiley.
- Detti, P., & Meloni, C. (2004). A linear algorithm for the hamiltonian completion number of the line graph of a cactus. *Discrete Applied Mathematics*, 136(2–3), 197–215.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. New York: Freeman.
- Goodman, S. E., Hedetniemi, S. T., & Slater, P. J. (1975). Advances on the hamiltonian completion problem. *Journal of the ACM*, 22(3), 352–360.
- Graham, R. L., Lawler, E. L., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1979). Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*, 5, 287–326.

- Hung, R.-W., & Chang, M.-S. (2006). Solving the path cover problem on circular-arc graphs by using an approximation algorithm. *Discrete Applied Mathematics*, 154(1), 76–105.
- Hung, R.-W., & Chang, M.-S. (2007). Finding a minimum path cover of a distance-hereditary graph in polynomial time. *Discrete Applied Mathematics*, 155(17), 2242–2256.
- Kundu, S. (1976). A linear algorithm for the hamiltonian completion number of a tree. *Information Processing Letters*, 5, 55–57.
- Lehoux-Lebacque, V., Brauner, N., & Finke, G. (2009). Identical coupled task scheduling: polynomial complexity of the cyclic case. *Les Cahiers Leibniz*, 179.
- Moran, S., & Wolfstahl, Y. (1988). Optimal covering of cacti by vertex-disjoint paths. *Theoretical Computer Science*, 84, 179–197.
- Nakano, K., Olariu, S., & Zomaya, A. Y. (2003). A time-optimal solution for the path cover problem on cographs. *Theoretical Computer Science*, 290(3), 1541–1556.
- Orman, A. J., & Potts, C. N. (1997). On the complexity of coupled-task scheduling. *Discrete Applied Mathematics*, 72, 141–154.
- Schrijver, A. (2004). *Combinatorial optimization: polyhedra and efficiency (algorithms and combinatorics)*. Berlin: Springer.
- Shapiro, R. D. (1980). Scheduling coupled tasks. *Naval Research Logistics Quarterly*, 27, 477–481.
- Simonin, G. (2009). Impact du graphe de compatibilité sur la complexité et l'approximation des problèmes d'ordonnancement en présence de tâches-couplées. *Ph.D thesis, LIRMM*, December 2009.
- Simonin, G., Giroudeau, R., & König, J.-C. (2009a). Complexity and approximation for scheduling problem for a torpedo. In *CIE'39: The 39th international conference on computers and industrial engineering, IEEE*, Troyes, France (pp. 300–304).
- Simonin, G., Giroudeau, R., & König, J.-C. (2009b). Extended matching problem for a coupled-tasks scheduling problem. In *TM-FCS'09: international conference on theoretical and mathematical foundations of computer science*, Orlando, Florida (pp. 082–089).
- Srikant, R., Sundaram, R., Sher Singh, K., & Rangan, P. C. (1993). Optimal path cover problem on block graphs and bipartite permutation graphs. *Theoretical Computer Science*, 115(2), 351–357.